← Back to calendar

20% progress of Week 1

Mandatory deliverable

LAB | Setup Nginx Server Lab

Skills you will learn

Version Control & Linux: Linux Fundamentals

1. Introduction

Nginx is a powerful, high-performance web server that is widely used as a **reverse proxy, load balancer, and caching server**. It is designed to handle large volumes of traffic efficiently, making it an essential tool for modern web hosting and system administration.

In this lab, you will learn how to **install, configure, and manage an Nginx web server on an Ubuntu Linux virtual machine**.

2. Requirements

- **Ubuntu Linux Virtual Machine**
- **Administrative privileges** (sudo access)
- **Port 80 availability** (Ensure no other service like Apache or another Nginx instance is using port 80)

To stop conflicting services before proceeding, run:

```
sudo systemctl stop apache2
sudo systemctl stop nginx
```

Copy Hide explanation

This snippet is about freeing up port 80 so Nginx can run without conflict. The key idea is that only one web server should usually listen on the same port at a time, so you stop any existing service that might already be using it.

`apache2` is another web server, and `nginx` is the service you want to manage in this lab. Stopping both ensures there isn't an old Apache process or a previously running Nginx instance still holding the port open. That prevents the common "address already in use" problem when you later start or install Nginx.

A useful mental model: before starting a server, make sure nothing else is already “standing at the door” on that port. If you skip this and another service is active, Nginx may fail to start or behave unexpectedly.

WAS THIS HELPFUL?

AI TUTOR ACTIONS

Ask a follow-up Explain more simply Show another example

3. Installing Nginx

3.1 Updating Package Lists

Before installing Nginx, update the package lists to ensure you get the latest version:

```
sudo apt update -y
```

Copy Hide explanation

This command refreshes Ubuntu’s package lists so `apt` knows about the latest available software versions before you install anything. The main idea is: your machine first checks the package repositories for updated metadata, then uses that information to make better install decisions.

`sudo` runs the command with administrator privileges, which is required because updating system package information changes a protected part of the OS. `apt update` does not install or upgrade packages by itself; it only downloads the newest package index files. The `-y` flag is harmless here but not really needed for `apt update` because this command usually doesn’t ask for confirmation.

One useful mental model: `apt update` is like refreshing a catalog before shopping, so later install commands can find the right package versions.

WAS THIS HELPFUL?

AI TUTOR ACTIONS

Ask a follow-up Explain more simply Show another example

3.2 Installing Nginx

To install Nginx, run:

```
sudo apt install nginx -y
```

Copy Explain

This command will install **Nginx and all necessary dependencies**.

3.3 Verifying Nginx Installation

Once installed, Nginx should start automatically. Check its status:

```
sudo systemctl status nginx
```

Copy Explain

If Nginx is running, you should see output indicating that the service is **active**.

To test access, open a browser and visit:

> `http://<server_ip_address>`

If Nginx is running correctly, you should see the **default Nginx welcome page**.

4. Managing the Nginx Process

Use the following `systemctl` commands to manage Nginx:

- **Stop Nginx:**

```
sudo systemctl stop nginx
```

Copy Explain

- **Start Nginx:**

```
sudo systemctl start nginx
```

Copy Explain

- **Restart Nginx:**

```
sudo systemctl restart nginx
```

Copy Explain

- **Reload Nginx (apply configuration changes without restarting):**

```
sudo systemctl reload nginx
```

Copy Explain

- **Disable Nginx from starting on boot:**

```
sudo systemctl disable nginx
```

Copy Explain

- **Enable Nginx to start on boot:**

```
sudo systemctl enable nginx
```

Copy Explain

5. Setting Up a Website on Nginx

By default, Nginx serves content from `/var/www/html`. To host multiple websites, you need **custom server blocks (virtual hosts).**

5.1 Creating a New Website Directory

Create a directory for your new website:

```
sudo mkdir -p /var/www/your_domain/html
```

Copy | Hide explanation

This command creates the folder structure where Nginx will look for the website files. The key idea is that a web server needs a directory on disk to serve content from, and this path is being prepared for a custom site.

`mkdir` means "make directory," and the `-p` option makes the command create any missing parent folders along the way. So if `/var/www/your_domain` does not already exist, it will be created too, instead of failing. That makes the command safe and convenient when setting up a new site from scratch.

`sudo` is used because `/var/www` is a system-owned location, so normal users usually do not have permission to create folders there. A small gotcha: `your_domain` is just a placeholder name, not a real domain yet. You would replace it with the folder name you want to use for that website.

WAS THIS HELPFUL?

AI TUTOR ACTIONS

Ask a follow-up | Explain more simply | Show another example

Set the correct ownership:

```
sudo chown -R $USER:$USER /var/www/your_domain/html
```

Copy | Hide explanation

This command changes who owns the website files so your normal user account can edit them without needing `sudo` every time. In web server setups, ownership matters because Nginx only needs to read the files, but you usually want your own user to be the one managing the content.

`chown` means "change owner." The `-R` flag makes it recursive, so it applies to the folder and everything inside it, not just the top directory. `$USER:$USER` sets both the owner and the group to your current login user, which is a convenient way to make the site directory writable by you while still keeping the permissions consistent.

The key idea is: give yourself control over the website files, while Nginx simply serves them. A common gotcha is that if you skip this step, you may create files as `root` and then run into permission

problems later when editing the site.

WAS THIS HELPFUL?

AI TUTOR ACTIONS

Ask a follow-up | Explain more simply | Show another example

Ensure proper permissions:

```
sudo chmod -R 755 /var/www/your_domain
```

Copy | Explain

5.2 Creating an Index Page

Create an HTML file to serve as the homepage:

```
sudo nano /var/www/your_domain/html/index.html
```

Copy | Explain

Add the following content:

```
<html>
    <head>
        <title>Welcome to Your Domain!</title>
    </head>
    <body>
        <h1>Success! The your_domain server block is working!</h1>
    </body>
</html>
```

Copy | Explain

Save and exit the file.

5.3 Configuring a Server Block

Create a new Nginx configuration file:

```
sudo nano /etc/nginx/sites-available/your_domain
```

Copy | Explain

Paste the following configuration:

```
server {
    listen 80;
    listen [::]:80;
```

Copy | Explain

```
    root /var/www/your_domain/html;
    index index.html index.htm;

    server_name _;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

Save and exit.

5.4 Enabling the Server Block

Enable the new configuration by creating a symbolic link to `sites-enabled`:

```
sudo ln -s /etc/nginx/sites-available/your_domain /etc/nginx/sites-enabled/
```

Copy | Hide explanation

This command connects a site configuration to Nginx's active configuration set. In Nginx, `sites-available` is where configuration files are stored, but `sites-enabled` is what Nginx actually reads when it starts or reloads. So this step is how you "turn on" a website without copying the file.

`ln -s` creates a symbolic link, which is like a shortcut pointing from `/etc/nginx/sites-enabled/your_domain` to the real file in `/etc/nginx/sites-available/your_domain`. That means you keep one source of truth for the config, and enabling or disabling a site becomes as simple as adding or removing that link.

One gotcha: if a link with the same name already exists, this command will fail. Also, Nginx won't use the new site until you test the config and reload or restart the service.

WAS THIS HELPFUL?

AI TUTOR ACTIONS

Ask a follow-up | Explain more simply | Show another example

Check for configuration errors:

```
sudo nginx -t
```

Copy | Explain

If everything is correct, you should see:

```
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Restart Nginx for changes to take effect:

```
sudo systemctl restart nginx
```

Copy Explain

Nginx should now be serving **your custom domain**.

IRONHACK

- Installing Nginx.
- Verifying the installation.
- Configuring and testing the server block.

2. Compile screenshots into a **Google Doc or PDF**.
3. Upload the document to **Google Drive** and share the link with your instructor.
4. Ensure the instructor has the necessary **viewing permissions**.

🎉 **Congratulations!** You have successfully installed and configured **Nginx on an Ubuntu Linux machine**! 🚀

Lab completed!

Submitted on Sept 18, 2026 a las 15:05

docs.google.com/document/d/1zx2lE_Nc8qRW--OlO6Y...

Edit lab

Progress of Week 1 10/49

You have 39 units left to complete

Mandatory deliverables 10/16

Skills learned +28% average

Skill: Version Control & Linux +28% 28%